Minería de Datos - Libro 1 - Clase 1 a 10

5. Limpieza y Transformacion Avanzada

Clase N° 5: Limpieza y Transformación de Datos en Minería de Datos

1. PRESENTACIÓN

Objetivo de la Clase

El preprocesamiento de datos es una fase crítica en la minería de datos. Antes de aplicar modelos de Machine Learning, debemos asegurarnos de que los datos sean limpios, consistentes y bien estructurados.

En esta clase, aprenderemos a:

- Detectar y manejar datos faltantes usando estrategias como eliminación e imputación.
- Identificar y tratar outliers con métodos estadísticos.
- Transformar datos mediante normalización, estandarización y codificación de variables categóricas.
- Usar Python y bibliotecas como Pandas y Scikit-learn para preprocesar datos de manera eficiente.

Importancia del Preprocesamiento de Datos

- Datos sucios pueden generar errores en los análisis y modelos predictivos.
- La transformación correcta de los datos mejora la precisión de los modelos de Machine Learning.
- Algunas técnicas como normalización y estandarización ayudan a que los modelos converjan más rápido y funcionen mejor.

Ejemplo real: Un hospital quiere predecir el tiempo de recuperación de pacientes basándose en su edad, tipo de tratamiento y otros factores. Si los datos tienen valores faltantes en edad o tipo de tratamiento, los resultados serán poco confiables o sesgados.

Sección 1: Manejo de Datos Faltantes

¿Por qué aparecen valores faltantes en los datos?

Los datos faltantes pueden deberse a varias razones:

- **Errores de entrada:** Un usuario no completó un campo en un formulario.
- **Fallas en la recolección de datos:** Sensores que fallan al registrar información.
- **Filtros o reglas de negocio:** Algunas variables solo se registran bajo ciertas condiciones.

1.1. Estrategias para manejar datos faltantes

Estrategia	Descripción	Cuándo Usarla	Ventajas	Desventajas
Eliminar filas/columnas	Descarta datos incompletos.	Cuando los valores faltantes son pocos y aleatorios.	Fácil de aplicar.	Puede eliminar información útil.
Imputación con media/mediana/moda	Rellena valores faltantes con la media, mediana o moda de la columna.	Cuando los valores faltantes son pocos y los datos siguen una distribución estable.	Preserva la cantidad de datos.	Puede sesgar los resultados si hay muchos valores faltantes.
Imputación con valores específicos	Sustituye valores faltantes con un valor arbitrario (ej. "Desconocido").	Cuando queremos evitar perder datos sin afectar la distribución.	Mantiene la estructura del dataset.	Puede introducir información no realista.
Modelos de imputación (KNN, regresión, MICE)	Predice los valores faltantes basándose en otras variables.	Cuando hay un patrón predecible en los datos.	Mantiene consistencia en los datos.	Requiere mayor computación y validación.

1.2. Ejemplo en Python – Manejo de Datos Faltantes

```
import pandas as pd
import numpy as np

# Crear DataFrame con valores faltantes
df = pd.DataFrame({
    "Nombre":  ["Ana", "Juan", "Pedro", "María", "Luis"],
    "Edad":    [25, np.nan, 40, np.nan, 30],
    "Salario": [50000, 60000, np.nan, 55000, 45000]
})
print("Datos Originales:
", df)

# Imputación con la mediana
df["Edad"].fillna(df["Edad"].median(), inplace=True)

# Eliminación de filas con valores nulos
df.dropna(subset=["Salario"], inplace=True)
print("
Datos después del Preprocesamiento:
", df)
```

Conclusión:

- La imputación con la mediana preserva la información de la variable Edad.
- Eliminar filas con valores nulos en Salario evita introducir datos incorrectos.

Ejercicio para la Sección 1

Objetivo: Aplicar técnicas de manejo de valores faltantes en un dataset real.

Instrucciones:

1. Carga el dataset Titanic de Seaborn (`sns.load_dataset("titanic")`).
2. Identifica qué columnas tienen valores nulos y cuántos valores faltantes hay en cada una.
3. Elimina las filas con valores nulos en la columna `age`.
4. Imputa los valores faltantes en la columna `embark_town` usando la moda.
5. Explica por qué elegiste esas estrategias y qué impacto pueden tener en un análisis posterior.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd

# Cargar dataset Titanic
df = sns.load_dataset("titanic")

# Ver cantidad de valores nulos por columna
print("Valores nulos por columna:
", df.isnull().sum())

# 1. Eliminar filas con valores nulos en 'age'
df_cleaned = df.dropna(subset=["age"])

# 2. Imputar valores nulos en 'embark_town' con la moda
df_cleaned["embark_town"].fillna(df_cleaned["embark_town"].mode()[0], inplace=True)

print("
Valores nulos después del preprocesamiento:
", df_cleaned.isnull().sum())
```

Preguntas de Reflexión

1. ¿Cómo afecta la eliminación de datos a la cantidad de información disponible?
2. ¿Qué problemas pueden surgir si imputamos valores en variables categóricas?
3. ¿Cómo podríamos validar que nuestras imputaciones no introducen sesgos?

Sección 2: Detección y Tratamiento de Outliers

Objetivo: Identificar valores atípicos en un conjunto de datos y aplicar técnicas adecuadas para manejarlos.

2.1. ¿Qué son los Outliers y por qué Importan?

- Un **outlier** (valor atípico) es un punto de datos que se aleja significativamente del resto de las observaciones.
- Los outliers pueden deberse a errores de medición, errores de entrada o fenómenos reales poco comunes.

Ejemplo de un outlier en la vida real: Si analizamos los salarios de empleados en una empresa, pero encontramos que un solo empleado gana 100 veces más que los demás, ese dato podría ser un error o una excepción justificada (ejemplo: CEO de la empresa).

2.2. Métodos para Detectar Outliers

Método	Descripción	Cuándo Usarlo
Boxplot	Identifica valores atípicos fuera del rango intercuartil.	Datos numéricos con distribución normal.
Z-score	Detecta outliers midiendo la desviación estándar desde la media.	Datos con distribución normal.
IQR (Rango Intercuartil)	Considera valores fuera de 1.5 veces el rango intercuartil como outliers.	Datos con distribución sesgada o no normal.
DBSCAN	Detecta outliers en datos espaciales y de clustering.	Cuando los outliers son puntos alejados en múltiples dimensiones.

2.3. Ejemplo en Python – Detección de Outliers con Boxplot y Z-score

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import zscore

# Crear DataFrame con un outlier
df = pd.DataFrame({
    "Empleado": ["A", "B", "C", "D", "E", "F", "G"],
    "Salario":  [50000, 52000, 54000, 58000, 60000, 62000, 250000]  # 250000 es un outlier
})

# Boxplot para visualizar outliers
sns.boxplot(x=df["Salario"])
plt.title("Boxplot de Salarios")
plt.show()

# Detectar outliers con Z-score
df["Z_Score"] = zscore(df["Salario"])
outliers = df[df["Z_Score"].abs() > 3]
print("Outliers detectados:
", outliers)
```

Conclusión:

- Los boxplots ayudan a visualizar valores extremos de forma gráfica.
- El Z-score nos permite identificar valores que se alejan más de 3 desviaciones estándar de la media.

2.4. Estrategias para Manejar Outliers

Estrategia	Descripción	Cuándo Usarla
Eliminar el outlier	Se descarta el dato si se confirma que es un error.	Cuando el outlier es un error evidente.
Transformación logarítmica	Se usa logaritmo para reducir el impacto de valores extremos.	Cuando los outliers afectan la distribución de datos.
Winsorización	Sustituye valores extremos con percentiles más cercanos.	Cuando queremos reducir el impacto sin eliminar datos.
Modelos robustos	Algunos algoritmos (ej. regresión robusta) pueden manejar outliers sin eliminarlos.	Cuando los outliers son parte del fenómeno real.

Ejercicio para la Sección 2

Objetivo: Aplicar detección y tratamiento de outliers en un dataset real.

Instrucciones:

1. Carga el dataset Diamonds de Seaborn (`sns.load_dataset("diamonds")`).

2. Crea un boxplot para visualizar outliers en la columna `price`.
3. Aplica el método IQR para detectar y eliminar valores atípicos en `price`.
4. Explica qué impacto tendría eliminar estos valores en un modelo de Machine Learning.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Cargar dataset
df = sns.load_dataset("diamonds")

# Boxplot de precios
sns.boxplot(x=df["price"])
plt.title("Boxplot de Precios de Diamantes")
plt.show()

# Cálculo del IQR
Q1 = df["price"].quantile(0.25)
Q3 = df["price"].quantile(0.75)
IQR = Q3 - Q1

# Definir límites para detectar outliers
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

# Filtrar valores dentro de los límites
df_cleaned = df[(df["price"] >= limite_inferior) & (df["price"] <= limite_superior)]
print("Antes:", len(df), "Después:", len(df_cleaned))
```

Preguntas de Reflexión

1. ¿Cómo cambia la distribución de `price` después de eliminar outliers?
2. ¿Eliminar outliers mejora siempre un análisis? ¿Cuándo puede ser contraproducente?
3. ¿Existen casos en los que los outliers pueden ser la parte más importante de los datos?

Sección 3: Normalización, Estandarización y Codificación de Datos Categóricos

Objetivo: Aplicar técnicas de transformación de datos numéricos y categóricos para mejorar la calidad de los modelos de minería de datos.

3.1. Normalización y Estandarización

¿Por qué transformar los datos?
Algunos algoritmos de Machine Learning son sensibles a la escala de los datos. Si una variable tiene valores muy grandes en comparación con otra, puede dominar el modelo e influenciar los resultados.

Diferencias entre Normalización y Estandarización

Técnica	Fórmula	Rango de salida	Cuándo usarla
Normalización (Min-Max Scaling)	`X_norm = (X - X_min) / (X_max - X_min)`	[0, 1]	Cuando los datos no siguen una distribución normal.
Estandarización (Z-score Scaling)	`X_std = (X - μ) / σ`	Centrado en 0 con desviación estándar 1	Cuando los datos tienen una distribución normal.

Ejemplo en Python – Aplicando Normalización y Estandarización

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Crear DataFrame con datos ficticios
df = pd.DataFrame({
    "Edad":    [18, 25, 30, 35, 40, 50, 60, 70],
    "Salario": [25000, 32000, 40000, 50000, 60000, 75000, 90000, 120000]
})

# Normalización (Min-Max Scaling)
scaler_minmax = MinMaxScaler()
df["Salario_Normalizado"] = scaler_minmax.fit_transform(df[["Salario"]])

# Estandarización (Z-score)
scaler_standard = StandardScaler()
df["Salario_Estandarizado"] = scaler_standard.fit_transform(df[["Salario"]])

print("
Datos transformados:
", df)
```

Conclusión:

- La normalización ajusta los datos al rango [0,1], útil para modelos como redes neuronales.
- La estandarización transforma los datos con media 0 y desviación estándar 1, ideal para algoritmos sensibles a la distribución normal como regresión logística.

3.2. Codificación de Variables Categóricas

¿Por qué transformar variables categóricas?
Los algoritmos de Machine Learning no pueden interpretar directamente datos categóricos. Debemos convertir estas variables a valores numéricos.

Métodos de Codificación

Técnica	Descripción	Ejemplo
Label Encoding	Asigna un número único a cada categoría.	`{"Rojo": 0, "Verde": 1, "Azul": 2}`
One-Hot Encoding	Crea una columna binaria para cada categoría.	`{"Rojo": [1,0,0], "Verde": [0,1,0], "Azul": [0,0,1]}`
Ordinal Encoding	Asigna valores según un orden lógico.	`{"Bajo": 1, "Medio": 2, "Alto": 3}`

Ejemplo en Python – Label Encoding y One-Hot Encoding

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Crear DataFrame con variables categóricas
df = pd.DataFrame({"Color": ["Rojo", "Verde", "Azul", "Verde", "Rojo"]})

# Aplicar Label Encoding
le = LabelEncoder()
df["Color_Label"] = le.fit_transform(df["Color"])

# Aplicar One-Hot Encoding
df_ohe = pd.get_dummies(df["Color"], prefix="Color")

print("
Label Encoding:
", df)
print("
One-Hot Encoding:
", df_ohe)
```

Conclusión:

- Label Encoding es útil para modelos de árboles de decisión, pero introduce un orden que puede no ser real.
- One-Hot Encoding es mejor para modelos lineales, pero aumenta la dimensionalidad del dataset.

Ejercicio para la Sección 3

Objetivo: Aplicar transformación de datos numéricos y categóricos en un dataset real.

Instrucciones:

1. Carga el dataset Titanic de Seaborn (`sns.load_dataset("titanic")`).
2. Aplica normalización Min-Max en la columna `fare`.
3. Convierte la columna `sex` a valores numéricos usando One-Hot Encoding.
4. Explica qué impacto puede tener esta transformación en un modelo de clasificación.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Cargar dataset Titanic
df = sns.load_dataset("titanic")

# Normalización de la columna fare
scaler = MinMaxScaler()
df["fare_normalizado"] = scaler.fit_transform(df[["fare"]])

# One-Hot Encoding de la columna sex
df = pd.get_dummies(df, columns=["sex"], prefix="Sexo")

print("
Datos transformados:
", df[["fare", "fare_normalizado", "Sexo_female", "Sexo_male"]].head())
```

Preguntas de Reflexión

1. ¿Por qué One-Hot Encoding es mejor que Label Encoding en este caso?
2. ¿Cómo cambiaría el rendimiento de un modelo si usamos datos sin normalizar?
3. ¿Cuándo puede ser preferible usar una codificación ordinal?

Sección 4: Ejercicio Integrador y Publicación en el Foro

Objetivo: Aplicar todas las técnicas de limpieza y transformación de datos vistas en la clase a un dataset real y fomentar la discusión en el foro.

4.1. Ejercicio Integrador

Supongamos que trabajamos en una empresa de análisis financiero y tenemos un dataset con información de clientes bancarios. Nuestro objetivo es preparar los datos para un modelo de predicción de solvencia financiera.

Dataset para el ejercicio

Cliente	Edad	Ingresos	Historial de Pagos	Categoría
1	25	50000	Bueno	A
2	40	60000	Regular	B
3	35	70000	Malo	C
4	50	80000	Bueno	A
5	30	45000	Regular	B
6	45	85000	Malo	C
7	38	72000	Bueno	A
8	29	48000	Regular	B

Tareas a realizar

1. **Limpieza de Datos**
   - Identificar y manejar valores nulos en Edad o Ingresos.
   - Detectar outliers en Ingresos usando el método IQR y decidir si eliminarlos o transformarlos.
2. **Transformación de Datos Numéricos**
   - Normalizar Ingresos usando Min-Max Scaling.
   - Estandarizar Edad con Z-score Scaling.

3. **Transformación de Datos Categóricos**
   - Aplicar One-Hot Encoding a la columna Historial de Pagos.
   - Aplicar Label Encoding a la columna Categoría.
4. **Interacción con una Herramienta de IA (LLM)**
   - Hacer una consulta a ChatGPT, Gemini u otra herramienta LLM sobre técnicas avanzadas de preprocesamiento de datos.
   - Ejemplo de consulta: "*¿Qué métodos avanzados de imputación de datos recomiendas para conjuntos de datos con muchas variables numéricas y categóricas?*"
   - Comparar la respuesta con lo aprendido en la clase y reflexionar sobre su utilidad.

Código Base para la Solución

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder

# Crear DataFrame
df = pd.DataFrame({
    "Cliente":          [1, 2, 3, 4, 5, 6, 7, 8],
    "Edad":             [25, 40, 35, 50, 30, 45, 38, 29],
    "Ingresos":         [50000, 60000, 70000, 80000, 45000, 85000, 72000, 48000],
    "Historial_Pagos":  ["Bueno", "Regular", "Malo", "Bueno", "Regular", "Malo", "Bueno", "Regular"],
    "Categoría":        ["A", "B", "C", "A", "B", "C", "A", "B"]
})

# 1) Manejo de valores nulos
df.fillna(df.median(numeric_only=True), inplace=True)

# 2) Detección de outliers con IQR
Q1 = df["Ingresos"].quantile(0.25)
Q3 = df["Ingresos"].quantile(0.75)
IQR = Q3 - Q1
df_cleaned = df[(df["Ingresos"] >= Q1 - 1.5*IQR) & (df["Ingresos"] <= Q3 + 1.5*IQR)]

# 3) Transformación numérica
scaler_minmax = MinMaxScaler()
df_cleaned["Ingresos_Normalizado"] = scaler_minmax.fit_transform(df_cleaned[["Ingresos"]])

scaler_std = StandardScaler()
df_cleaned["Edad_Estandarizada"] = scaler_std.fit_transform(df_cleaned[["Edad"]])

# 4) Transformación categórica
df_cleaned = pd.get_dummies(df_cleaned, columns=["Historial_Pagos"], prefix="Historial")
le = LabelEncoder()
df_cleaned["Categoría_Label"] = le.fit_transform(df_cleaned["Categoría"])

print("
Datos transformados:
", df_cleaned)
```

Instrucciones para compartir resultados

Publica un mensaje en el foro obligatorio de la clase con los siguientes puntos:

- Un resumen del proceso de limpieza y transformación de datos.
- Capturas o gráficos que muestren antes y después de la limpieza y transformación.
- ¿Cómo afectó la normalización y estandarización a los datos?
- ¿Qué ventajas y desventajas observaste en el uso de One-Hot Encoding vs. Label Encoding?
- ¿Qué insights obtuviste usando una herramienta LLM (ChatGPT, Gemini, etc.)?

Ejemplo de Publicación en el Foro

> **Resumen del Preprocesamiento:** "Se identificaron y eliminaron outliers en la columna de ingresos. Luego, se aplicó normalización Min-Max en Ingresos y estandarización Z-score en Edad. Finalmente, se usó One-Hot Encoding para Historial de Pagos y Label Encoding para Categoría."
>
> **Resultados Visuales:** "Antes de la transformación, los ingresos tenían una gran variabilidad. Después de la normalización, todos los valores se encuentran entre 0 y 1, lo que facilitará la convergencia de modelos de Machine Learning."
>
> **Consulta a ChatGPT:** "Pregunté a ChatGPT sobre imputación avanzada de valores nulos y recomendó probar el método KNN Imputer. Implementé esta estrategia y observé que la predicción es más precisa cuando las variables tienen correlación significativa."

4.3. Participación en la Publicación de un Compañero

Comenta en la publicación de otro estudiante:

- **Pregunta:** ¿Cómo cambiarían los resultados si usáramos otro método de imputación de valores nulos?
- **Comparación:** ¿Coincide la respuesta del LLM con lo que observaste en tu propio análisis?
- **Sugerencias:** ¿Cómo podríamos mejorar la transformación de datos?

Reflexión Final de la Clase

- ¿Cómo impactó la limpieza y transformación de datos en la calidad del dataset?
- ¿Qué técnicas de preprocesamiento resultaron más efectivas?
- ¿Qué problemas pueden surgir si los datos no se limpian correctamente?

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier.
2. McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
4. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
6. Wickham, H., & Grolemund, G. (2016). *R for Data Science*. O'Reilly Media.
7. Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.